

MANUAL DE OPERACIÓN E INSTALACIÓN

Sistema de Control de Accesos Banco Santander / SAIMA SEGURIDAD

Sistema local (panel de oficina) · Centro de Control (COCE) · App Android (tablet)

Versión documental: 1.1 — 05/10/2026

Documento orientado a instalación, configuración, uso operativo y actualizaciones.

Incluye tres bloques independientes: Sistema local, COCE y App Android.

Índice

1. Visión general del sistema
2. Arquitectura y comunicaciones
3. Puertos y servicios
4. SISTEMA LOCAL (PC industrial de oficina)
5. CENTRO DE CONTROL (COCE)
6. APP ANDROID (tablet Akuvox)
7. Actualizaciones remotas (COCE → sucursales)
8. Modos operativos y funcionamiento
9. Zaguán, llamadas a tablet e intercom
10. Integración Panphone / CSIP (opcional)
11. Checklist de puesta en marcha
12. Solución de problemas frecuentes
13. Anexos (variables de entorno y rutas)

1. Visión general del sistema

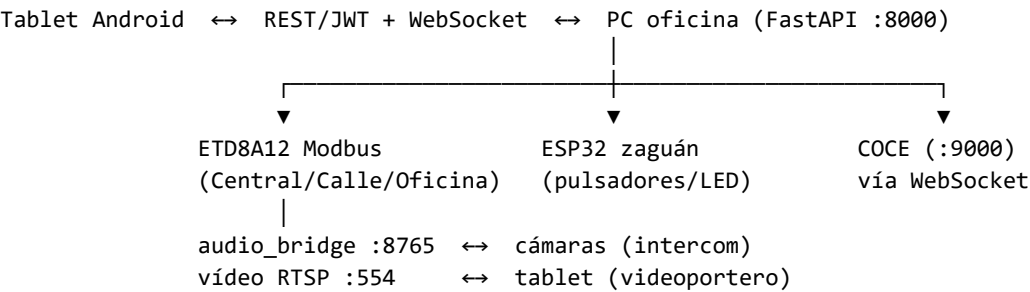
El sistema de control de accesos gestiona las puertas de zaguán de oficinas del Banco Santander. Se organiza en tres piezas de software que colaboran entre sí:

Pieza	Dónde corre	Función
Sistema local (panel)	PC industrial de cada oficina	Lógica de modos, Modbus a placas ETD8A12, API tablet, panel web, zaguán ESP32, enlace a COCE
COCE (API + dashboard)	PC/servidor del Centro de Control	Supervisión de sucursales, mensajes, técnicos, cambio de modo remoto, actualizaciones
App Android	Tablet Akuvox en oficina	Selección de modo, apertura, vídeo RTSP, intercom, recepción de llamadas y mensajes COCE

El COCE nunca habla directamente con las tablets ni con las placas Modbus. Siempre pasa por el backend de la sucursal. Las tablets solo hablan con el PC de oficina.

2. Arquitectura y comunicaciones

Flujo lógico resumido:



Capas:

- Capa UI: tablet Android + panel web del operador en el PC.
- Capa lógica: servicio Python (FastAPI) en el PC industrial.
- Capa hardware: 3 módulos ETD8A12 (Modbus TCP o RTU) + nodos ESP32 del zaguán.
- Capa central: COCE API + dashboard (supervisión multi-sucursal).

3. Puertos y servicios

Puerto	Servicio
8000	Backend FastAPI de sucursal (API + panel web en producción)
5173	Frontend panel en desarrollo (Vite)
5174	Dashboard COCE en desarrollo (Vite)
9000	API COCE central
8765	Puente de audio intercom (audio_bridge.py)
502 / 5000	Modbus TCP de placas ETD8A12 (según firmware/config)
COM7 @ 9600	Modbus RTU por defecto (si MODBUS_MODE=rtu)
80	API HTTP de ESP32 zaguán
8266	OTA TCP del firmware ESP32
554	RTSP de cámaras / videoportero

8090	API CSIP / Panphone (si se usa)
------	---------------------------------

Para acceso remoto entre COCE y oficinas (y tablets fuera de LAN) se usa típicamente ZeroTier u otra VPN. La lógica de la aplicación no cambia: solo la topología de red.

4. SISTEMA LOCAL (PC industrial de oficina)

Incluye el backend Python (FastAPI), el frontend del panel de configuración, la comunicación Modbus, el orquestador de zaguán y el cliente WebSocket hacia COCE.

4.1 Obtención / descarga

- Repositorio: <https://github.com/jorgesolano92/santander.git>
- Clonar en el PC industrial: git clone <url> y checkout de la rama de despliegue acordada.
- Actualizaciones posteriores de panel: preferiblemente desde COCE (zip remoto), sin git pull en oficina (ver sección 7).

4.2 Requisitos

- Windows (PC industrial / Windows IoT según contrato).
- Python 3.9+ (recomendado 3.11/3.12).
- Node.js 18+ (solo para construir el frontend).
- Red Ethernet local con IPs estáticas para PC, placas y cámaras.
- Adaptador RS-485 o acceso TCP a las 3 placas ETD8A12.

4.3 Instalación (desarrollo)

Backend:

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python run.py
```

URL: <http://localhost:8000> — Documentación OpenAPI: <http://localhost:8000/docs>

Frontend (panel web):

```
cd frontend
npm install
npm run dev
```

URL: <http://localhost:5173> (Vite hace proxy de /api al puerto 8000).

4.4 Instalación / arranque en producción

Se construye el frontend y el backend sirve la SPA y la API en el mismo origen (:8000):

```
cd frontend
npm install
npm run build

cd ..\backend
venv\Scripts\activate
python run.py
# equivalente: uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Opcional: copiar frontend/dist a backend/static y definir STATIC_DIR=./static en .env. En producción el proceso debería ejecutarse como servicio Windows (NSSM, WinSW o similar) para arranque automático; el repositorio documenta el reinicio del servicio tras updates, aunque el instalador MSI/servicio no forma parte del paquete de código actual.

4.5 Configuración esencial (.env)

Archivo: backend/.env (partir de backend/.env.example). Principales variables:

Variable	Uso
DATABASE_URL	SQLite local (p.ej. sqlite:///./data/control_accesos.db)
MODBUS_MODE	tcp o rtu
MODBUS_SERIAL *	Puerto COM, baudios, etc. (si RTU)
TABLET_JWT_SECRET / PANEL_JWT_SECRET	Secretos JWT (obligatorios en producción)
TABLET_JWT_EXPIRE_MINUTES	Caducidad token tablet (p.ej. 10080 = 7 días)
COCE_WS_ENABLED / COCE_WS_URL	Enlace saliente al Centro de Control
COCE_INSTALLATION_ID / COCE_INGEST_TOKEN	Identidad de la sucursal en COCE
COCE_MESSAGE_SEND_WEB / TABLET	Reparto de mensajes a panel y/o tablets
ZAGUAN *	IP ESP32, captura-only, etc.
CSIP *	Integración Panphone (opcional)
TABLET_CALL *	Llamadas entrantes a tablets

La configuración de placas, reglas, horarios y zaguán se gestiona también desde el panel web y se persiste en SQLite (backend/data/control_accesos.db). Overrides de ESP32 por canal:
backend/data/zaguan_device_target.json.

4.6 Primer acceso al panel web

- Abrir http://<IP-PC>:8000 (producción) o :5173 (desarrollo).
- Registrar el primer usuario de panel (flujo de setup / PANEL_SETUP_TOKEN según .env).
- Configurar las 3 placas ETD8A12 (IP/puerto o slave_id), reglas y modos.
- Verificar lectura de entradas/salidas en el panel.

4.7 Actualización del sistema local

Opción A — Manual en el PC:

- git pull (o copiar release).
- Actualizar dependencias Python/npm si cambian.
- npm run build en frontend.
- Reiniciar el servicio / proceso Python.

Opción B — Remota desde COCE (recomendada en operación):

- COCE publica release kind=panel.
- En el panel de sucursal aparece banner → Actualizar panel.
- Se descarga zip, se verifica SHA-256 y se aplica sin tocar data/, BD ni .env.
- Reiniciar servicio Windows y recargar el navegador.

4.8 Componentes internos del sistema local

Componente	Ruta / nota
------------	-------------

API FastAPI	backend/app/main.py
Rutas tablet JWT	/api/v1/... y WS /api/v1/ws/calls
Rutas panel	/api/panel/... y WS /api/panel/ws/live
Orquestador zaguán	backend/app/services/zaguan_orchestrator.py
Hub llamadas tablet	backend/app/services/tablet_call_hub.py
Hardware Modbus	backend/app/hardware/
Cliente COCE	backend/app/coce/
Módulo CSIP Panphone	backend/app/csip/
Firmware / OTA ESP32	backend/zaguan_*.ino, python_ota_update.py

5. CENTRO DE CONTROL (COCE)

El COCE es el punto central de supervisión. Consta de coce-api (FastAPI + SQLite) y coce-dashboard (React). Las sucursales mantienen un WebSocket saliente hacia el COCE.

5.1 Obtención / descarga

- Mismo repositorio santander (rama main): carpetas coce-api/ y coce-dashboard/.
- Se despliega en la máquina del Centro de Control (IP conocida por las oficinas).

Linux (Ubuntu):

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip
cd ~
git clone -b main https://github.com/jorgesolano92/santander.git
cd santander
```

5.2 Requisitos

- Python 3.9+ y Node.js 18+.
- Red alcanzable por las sucursales (ZeroTier/VPN o IP pública con reglas de firewall).
- Secretos fuertes: COCE_JWT_SECRET y COCE_SECRETS_KEY (Fernet).

5.3 Instalación — API

La configuración de la API está en el archivo coce-api/.env. Ese archivo no viene en el repositorio (contiene secretos): se crea copiando la plantilla coce-api/.env.example que está en la misma carpeta, y después se editan en él las variables COCE_JWT_SECRET, COCE_SECRETS_KEY y COCE_CORS_ORIGINS.

Linux (Ubuntu):

```
cd ~/santander/coce-api
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
nano .env          # editar variables (ver tabla y comandos de abajo)
python -m app.main
```

Windows:

```
cd coce-api
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
notepad .env       # editar variables (ver tabla y comandos de abajo)
python -m app.main
```

Variables a editar en coce-api/.env:

Variable	Qué poner
COCE_JWT_SECRET	Cadena larga aleatoria para firmar las sesiones del dashboard (comando abajo).
COCE_SECRETS_KEY	Clave Fernet para cifrar credenciales guardadas de las

	sucursales (comando abajo).
COCE_CORS_ORIGINS	URL desde la que se abre el dashboard, p.ej. http://<IP-COCE>:5174. Varias URLs separadas por comas, sin espacios.
COCE_SETUP_TOKEN	Opcional. Permite registrar usuarios cuando ya existe alguno (ver 5.5).

Generar los valores (con el entorno .venv activado; el mismo comando sirve en Linux y Windows) y pegar cada resultado en su variable:

```
# COCE_JWT_SECRET
python -c "import secrets; print(secrets.token_urlsafe(48))"

# COCE_SECRETS_KEY
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

Ejemplo de líneas resultantes en coce-api/.env:

```
COCE_JWT_SECRET=<resultado del primer comando>
COCE_SECRETS_KEY=<resultado del segundo comando>
COCE_CORS_ORIGINS=http://192.168.1.50:5174,http://localhost:5174
```

Importante: no cambiar COCE_SECRETS_KEY una vez el COCE esté en servicio; las credenciales de sucursal ya guardadas dejarían de poder descifrarse. Cambiar COCE_JWT_SECRET solo obliga a volver a iniciar sesión en el dashboard.

Escucha por defecto en http://0.0.0.0:9000 — Health: http://<host>:9000/health

Comprobación rápida en Linux:

```
curl http://localhost:9000/health
```

5.4 Instalación — Dashboard

Linux (Ubuntu):

```
cd ~/santander/coce-dashboard
npm install
cp .env.example .env
nano .env          # VITE_COCE_API_URL=http://<IP-COCE>:9000
npm run dev -- --host 0.0.0.0
```

Windows:

```
cd coce-dashboard
npm install
copy .env.example .env
# VITE_COCE_API_URL=http://localhost:9000
npm run dev
```

Desarrollo: http://localhost:5174

Producción:

```
npm run build
# servir dist/ con el mismo origen o CORS permitido
```

5.5 Primer usuario y sucursales

- Con BD vacía: registrar usuario vía UI o POST /api/coce/auth/register.
- Si ya hay usuarios: cabecera X-Coce-Setup-Token = COCE_SETUP_TOKEN.
- Login → JWT de sesión (dashboard en sessionStorage).
- Crear sucursales: se genera installation_id + ingest_token (mostrar una sola vez). Esos valores se copian al .env del backend de la oficina (COCE_INSTALLATION_ID, COCE_INGEST_TOKEN, COCE_WS_URL).

5.6 Funcionalidades operativas

Función	Descripción
Mapa / listado sucursales	Estado online/offline por heartbeat
Snapshot remoto	Modo activo y estado de placas vía proxy HTTP al panel
Cambio de modo remoto	POST set-mode con auditoría
Mensajería	Mensajes a sucursal → panel y/o tablets
Técnicos	Sincronización de técnicos hacia oficinas
Alertas	Eventos / avisos centralizados
Actualizaciones	Publicar panel zip o APK y desplegar por WS
Auditoría	Histórico de acciones administrativas (no editable desde sucursal)

5.7 Actualización del propio COCE

- En la máquina COCE: git pull origin main del repositorio.
- Actualizar dependencias; reiniciar coce-api.

Linux (Ubuntu):

```
cd ~/santander
git pull origin main
cd coce-api && source .venv/bin/activate && pip install -r requirements.txt
cd ../coce-dashboard && npm install && npm run build
```

- Rebuild del dashboard (npm run build) y redesplegar estáticos.
- No hay auto-update del COCE desde fuera; es el origen de los paquetes hacia sucursales.

5.8 Enlace sucursal ↔ COCE

En cada oficina (backend/.env):

```
COCE_WS_ENABLED=true
COCE_WS_URL=ws://<IP-COCE>:9000/api/coce/ws/branch/<installation_id>
COCE_INSTALLATION_ID=<uuid>
COCE_INGEST_TOKEN=<token>
COCE_HEARTBEAT_INTERVAL_SECONDS=60
COCE_MESSAGE_SEND_TABLET=true
COCE_MESSAGE_SEND_WEB=true
```

Tipos de mensajes por el canal WS (ejemplos): heartbeat, modos, coce_message, technicians_sync, software_update, update_status.

6. APP ANDROID (tablet Akuvox)

Aplicación Expo / React Native con módulos nativos Android (vídeo/SDK). Repositorio separado:
<https://github.com/jorgesolano92/santander-app.git>

6.1 Obtención / descarga

- Código fuente: clonar el repositorio santander-app.
- Instalación en tablets: APK firmada (debug o release) vía sideload / software Akuvox.
- Distribución operativa: COCE sube la APK → panel de oficina descarga → operador instala en tablets.

6.2 Requisitos de desarrollo

- Node.js 18+, npm.
- Android Studio / SDK / JDK para builds nativos.
- Dispositivo o emulador; para intercom nativo preferible tablet real.
- Opcional: EAS CLI para builds en la nube (eas.json).

6.3 Instalación en entorno de desarrollo

```
cd santander-app
npm install
npm run dev           # Expo
npm run android       # expo run:android (nativo)
npm run android:tablet # build/install orientado a tablet
```

6.4 Compilación de APK

Opción recomendada (Gradle, conserva código nativo SDK):

```
cd android
.\gradlew clean
.\gradlew assembleDebug
# o release:
.\gradlew assembleRelease

# APK debug:
# android\app\build\outputs\apk\debug\app-debug.apk
# Instalar:
.\gradlew installDebug
```

Con Expo:

```
npx expo run:android
```

Con EAS (perfiles en eas.json: apk-release, standalone, etc.):

```
eas build --platform android --profile apk-release
```

Importante: no usar comandos destructivos (git reset --hard) antes de compilar si hay cambios locales en el SDK nativo (ver docs/COMPILACION_ANDROID.md).

6.5 Configuración de la app

La configuración operativa vive en la tablet (AsyncStorage), con defaults en config/defaultDoorAppConfig.ts.
Campos clave:

- network.consoleIP — IP del PC industrial (ej. 192.168.1.155).
- api.port — 8000.
- Puertas: cámara IP, RTSP, bridgeUrl (ws://<PC>:8765), mapeo Modbus.
- Modos habilitados/deshabilitados por oficina.
- officeWithATM — muestra modo carga cajero si aplica.

Intercom actual en operación: modo puente PC (INTERCOM_BRIDGE_ONLY = true en config/intercomFeatures.ts). SIP/CSIP están preparados en código pero ocultos en UI hasta activarlos.

6.6 Puente de audio (PC de oficina)

Para intercom bidireccional con cámaras TVT/Safire se ejecuta en el PC (aparte del FastAPI):

```
cd Release_vcx_x64
pip install -r bridge_requirements.txt
python audio_bridge.py
```

- WebSocket en :8765.
- Requiere DLLs del SDK + Visual C++ Redistributable x64.
- Ajustes: BRIDGE_ENV.md y bridge_cameras.json.

6.7 Autorización de dispositivo

La app obtiene el Android ID de la tablet y consulta al backend si el dispositivo está autorizado. Si no lo está, muestra pantalla de dispositivo no autorizado. Las tablets deben estar dadas de alta / autorizadas en el panel o flujo configurado.

6.8 Funciones principales de la app

- Visualización del modo activo y descripción + esquema/diagrama.
- Cambio de modo (lista filtrada según modos habilitados en ajustes).
- Modo carga cajero: layout con vídeo a la derecha y descripción/esquema a la izquierda.
- Modo manual / bloqueo de puertas: control de aperturas e intercom.
- Llamadas entrantes (videoportero): suenan todas las tablets; la primera que contesta gana.
- Mensajes COCE (toast + historial).
- Emergencia e incendio (confirmación y desactivación).

6.9 Actualización de la app

Flujo operativo recomendado:

- Compilar y firmar APK en el entorno de desarrollo.
- En COCE → Actualizaciones: subir APK + versión (ej. 2.8.1) + notas.
- Lanzar despliegue a sucursales (kind=tablet_apk).
- En el panel de oficina: banner → Descargar APK.
- Instalar en cada tablet con el software Akuvox (no hay OTA silenciosa ni Play Store).

La tablet no descarga directamente del COCE. La versión la introduce el operador al publicar; no se lee automáticamente el versionCode del APK.

7. Actualizaciones remotas (COCE → sucursales)

Objetivo: distribuir software sin que las PCs de oficina tengan acceso a GitHub ni hagan git pull.

Tipo	kind	Qué actualiza	Acción en sucursal
Panel PC	panel	Backend + frontend	Banner → Actualizar panel → reiniciar servicio
APK tablet	tablet_apk	APK Android	Banner → Descargar APK → instalar con Akuvox

7.1 Publicar panel desde COCE

- En COCE: git pull del checkout del panel; opcional npm run build en frontend.
- En coce-api/.env: COCE_PANEL_SOURCE_DIR=<ruta al repo santander>.
- Dashboard → Actualizaciones → Publicar desde local (empaqueta backend + frontend/dist).
- Excluye: data/, *.db, .env, node_modules, .git, venv.
- Seleccionar release → Lanzar despliegue (todas o sucursales concretas).

7.2 Publicar APK

- Subir .apk + versión + changelog.
- Artefacto en coce-api/data/releases/ (fuera de git).
- Desplegar; las oficinas reciben aviso por WebSocket.

7.3 Estados reportados

La sucursal informa update_status: available | downloading | applying | success | failed | offline. El dashboard muestra la cola de despliegues.

8. Modos operativos y funcionamiento

Existen 7 modos con exclusión mutua (solo uno activo). Se activan por horario/entradas del módulo Central, por tablet, por panel o por COCE (remoto).

Modo	Idea operativa
Comercial automático	Flujo normal de atención en horario comercial
Comercial esclusa	Paso controlado tipo esclusa
Horario extendido	Fuera de horario comercial; llamadas tablet en ciertos pulsadores
Horario autoservicio	Autoservicio / acceso restringido según reglas
Oficina cerrada	Cierres de seguridad; puertas cerradas
Carga de cajero	Carga ATM en zaguán; P1 cerrada con llamada; vídeo en home
Bloqueo de puertas (manual)	Control manual desde tablet; llamadas a tablets

Además: emergencia e incendio (prioridad alta). El detalle de las 33 actuaciones (ATACA / ACTIVA / DESACTIVA / NO ACTUA SI) está en EXPLICACION_ACTUACIONES_Y_MODOS.md y en los Excel de actuaciones del proyecto.

9. Zaguán, llamadas a tablet e intercom

9.1 Pulsadores ESP32

El visitante pulsa un botón en el zaguán:

```
ESP32 → POST /zaguan/pulsacion/p1|p2|p3|p4
        → zaguan_orchestrator (según modo)
        → abre puerta por Modbus (si aplica) y/o inicia llamada tablet
        → actualiza LED del ESP32 (libre/ocupado/abriendo)
```

- p1 / p3 ≈ puerta calle (P1).
- p2 / p4 ≈ puerta oficina (P2).
- ZAGUAN_PULSACION_CAPTURE_ONLY=1: solo captura, no abre Modbus (banco de pruebas).

9.2 Llamadas a tablets (first-wins)

En modos configurados (p.ej. manual, carga cajero, extendido), la pulsación dispara `tablet_call_hub.start_call()`:

- Se envía `incoming_call` por WebSocket a TODAS las tablets conectadas.
- Todas suenan / muestran overlay.
- La primera que envía `answer` recibe `call_accepted`.
- Las demás reciben `call_taken` y dejan de sonar.
- Timeout configurable (`TABLET_CALL_TIMEOUT`).

Tras contestar, la tablet abre `ManualModeModal` y arranca el intercom del modo configurado (puente por defecto). El canal de intercom bidireccional también es exclusivo (`claim/release`): solo una tablet a la vez.

9.3 Intercom

Modo	Estado
bridge (puente PC)	Operación actual: tablet ↔ WS :8765 ↔ audio_bridge ↔ cámara
sdk	Nativo Android; disponible en código, UI condicionada
sip / CSIP	Preparados para Panphone + PBX; UI condicionada (<code>INTERCOM_BRIDGE_ONLY</code>)

10. Integración Panphone / CSIP (opcional)

Módulo backend aparte (`backend/app/csip/`) expuesto bajo `/api/csip`. Pensado para pruebas y futura operación con placa Panphone + centralita PBX.

- Webhooks: `POST /api/csip/notify, /notify/p1, /notify/p2` (`notification_url` en la placa).
- Proxy saliente: `call_start, led_control, button_event`.
- `CSIP_FORWARD_PULSACION_TO_ZAGUAN=true` para reutilizar la lógica de zaguán/llamadas tablet.

Patrón recomendado con PBX: la placa notifica al backend; suenan todas las tablets por WebSocket; solo la que contesta abre la sesión SIP / `call_start` hacia su extensión. Detalle técnico en `docs/CSIP_SIP_INTEGRATION.md` de la app.

11. Checklist de puesta en marcha

Orden sugerido:

- 1. Red local / ZeroTier: IPs de PC, placas, cámaras, ESP32, tablets.
- 2. Instalar y arrancar sistema local (backend + frontend build) en :8000.
- 3. Configurar Modbus (TCP o RTU) y verificar I/O en panel.
- 4. Flashear/configurar ESP32 zaguán y apuntar al backend; probar pulsaciones.
- 5. Arrancar audio_bridge :8765 si se usa intercom puente.
- 6. Instalar COCE (api :9000 + dashboard); crear usuario y sucursal.
- 7. Cablear COCE_WS_* en la oficina y verificar heartbeat online.
- 8. Compilar e instalar APK; apuntar consoleIP al PC; autorizar dispositivo.
- 9. Probar cambio de modo, apertura, llamada tablet, mensaje COCE.
- 10. Probar ciclo de actualización panel y APK desde COCE (banco).

12. Solución de problemas frecuentes

Síntoma	Comprobación
Tablet WS 403 / se desconecta	JWT caducado o inválido; revisar TABLET_JWT_* y re-login
No cambia modo / modo deshabilitado	Ajustes de tablet: modes.*.enabled; el modo no debe listarse si está off
Placa Modbus desconectada	Cableado RS-485/TCP, COM, slave_id, timeouts
COCE offline	COCE_WS_URL alcanzable, token e installation_id correctos
No suena llamada en tablets	Modo actual en TABLET_CALL_MODES; WS conectado; pulsador permitido
Sin audio intercom	audio_bridge en marcha :8765; bridgeUrl; DLLs/VC++
CSIP notify abre el panel en navegador	Normal en GET; el webhook es POST
Crash Android al abrir modos	SVG orient=auto-start-reverse; ya mitigado en Modelcon
Update panel no aplica	Banner apply; SHA; espacio disco; reinicio servicio tras apply

13. Anexos

13.1 Documentación de referencia en los repositorios

Documento	Contenido
santander/README.md	Arranque rápido
santander/DESARROLLO.md	Dev y build producción
santander/ARQUITECTURA.md	Capas y persistencia
santander/ACTUALIZACIONES_REMOTAS.md	Contrato updates COCE
santander/EXPLICACION_ACTUACIONES_Y_MODOS.md	Modos y actuaciones
santander/coce-api/README.md	API central
santander/coce-dashboard/README.md	UI COCE
santander/backend/SAIMA_*.md	ESP32 instalación/API
santander-app/docs/COMPILACION_ANDROID.md	Build APK
santander-app/docs/CSIP_SIP_INTEGRATION.md	SIP / CSIP
santander-app/Release_vcx_x64/BRIDGE_ENV.md	Puente audio

13.2 Rutas útiles

Panel oficina: <http://<IP-PC>:8000>
OpenAPI oficina: <http://<IP-PC>:8000/docs>

COCE API: http://<IP-COCE>:9000
COCE dashboard: http://<IP-COCE>:5174 (dev) o URL de producción
Zaguan ESP: http://<IP-ESP>/ ... OTA TCP :8266
Audio bridge: ws://<IP-PC>:8765

13.3 Límites actuales a tener en cuenta

- No hay Docker/compose ni MSI empaquetado en el repositorio.
- El servicio Windows se asume en operación pero el instalador no viene en el código.
- La APK no se autoinstala en Akuvox; requiere intervención del operador.
- SIP/Panphone: integración preparada; operación diaria actual usa puente PC.

— Fin del documento —

Control de Accesos Santander / SAIMA · Manual de operación e instalación v1.1